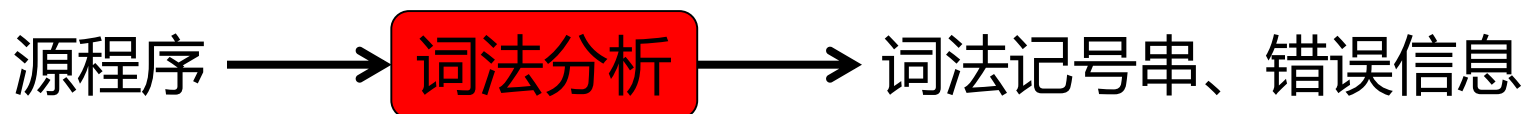


第四章 词法分析2026

赵伟

C2 词法分析



- 通过扫描源程序逐一识别出每个词法单元。
- 源程序是一个符号串，字母表一般为ASCII字符集；
- 词法记号串也是一个符号串，字母表是？每个词法记号都是某个词法单元的符号表示。
- 从**词法分析原理**反映出如何拆分源程序符号串、如何识别为词法单元、如何表示为词法记号。
- 能够手写或自动生成词法分析器。

知识图谱

- 源语言 L_{SRC} : (字母表 正则语言簇 $\mathcal{L}_{\text{eff}}$ 空白 注释 行号)
- 源程序: (词法单元 空白 注释 行号)
- 词法分析: (输入 输出 预处理 错误处理 扫描器: (硬编码 表驱动 自动生成) σ -DFA: (DFA $\mathcal{L} \mathbb{L} \psi$) 词法记号: (词法种属 词法单元))

- | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|--|---|--|---|---|---|--|---|---|---|---|---|--|--|--|--|--|--|
| 2 | 3 | 4 | | * | | (| 1 | 1 | | + | - | 2 | 2 |) | | | | | | |
|---|---|---|--|---|--|---|---|---|--|---|---|---|---|---|--|--|--|--|--|--|

- 词法分析任务：确定源程序w中每一个子串x及其所属语言。
- 示例确定出7个子串：234，*，（，11，+，-22，）
- 这些子串都是词法单元。
- 所属语言5个：NUM，MUL，LPA，ADD，RPA

记号的二元组表示

- 记号是类别和值组成的二元组，类别是语言的名字，由于所有语言都是已知的，所以可命名。特别地，单元素语言可以用其成员指代它。
- 值是词法单元的表现形式，可以是子串名、或子串值、或存储位置。
- 比如(NUM, 100)中的100可以是数值100也可是字符串“100”
- 比如(ID, x)中x可以是“x”也可以是x在符号表中的登记项。

记号的设计

- 全体一种：把某个符号串集合命名为语言KIND，它的每个元素w是一个类别为KIND的词法单元。词法记号可为(KIND, x)
- 对于单元素语言KIND_1，记号表示中值可省略，称为“一符一种”表示，即(KIND_1, _)。
- 对于元素难以枚举的语言KIND，比如数、标识符等，只能设计为“全体一种”的记号，即(KIND, <词法单元>)。
- 对于易枚举的语言，采用两种设计均可，取决于效果：
 - 关键字 (WHILE, _) (KEY, while)
 - 运算符 (LE, _) (ROP, <=)
 - 括号 (LBR, _) (BRACE, [)

常见记号的类别

类别	类别名字	语言定义
Identifiers	ID	<code>[a-zA-Z_][a-zA-Z_0-9]*</code>
Integer constants	NUM	<code>[+-]?[0-9]+</code>
Character constants	CHR	<code>'([a-zA-Z0-9] \\[a-zA-Z])'</code>
Floating constants	FLO	<code>[+-]?[0-9]*.[0-9][0-9]* ([+-]?[0-9][0-9]*.[0-9][0-9]*)?</code>
String constants	STR	<code>"([a-zA-Z0-9] \\[a-zA-Z])*"</code>
Operator tokens	ADD, MUL, ...	<code>+, *, <, <=, ==, ^, v, ¬, ...</code>
Assignments	ASG, AAS, ...	<code>=, +=, ++, ...</code>
括号	LPA, LBK, ...	<code>(), [], { }, ..</code>
Keywords	IF, INT, ...	<code>while, int, if, else, return, ...</code>

种属	种属标识	定义或列举
标识符	ID	[a-zA-Z][_a-zA-Z0-9]*
整数	NUM (INT_LITERAL), HEX_LITERAL	[+-]?[0-9]+, ...
字符常数	CHR_LITERAL	'[a-zA-Z0-9]'
浮点数	FLO (FLOAT_LITERAL), DOUBLE	{数学定点数} ([eE]{整数})?
字符串常数	STR_LITERAL	"[a-zA-Z0-9]*"
算术运算符	ADD (PLUS), SUB (MINUS), MUL, DIV, MOD, AAA (INC), DEC	<code>+, -, *, /, %, ++, --</code>
关系运算符	LT, LE, GT, GE, EG, NE	<code><, <=, >, >=, ==, !=</code>
布尔运算符	AND, OR, NOT	<code>^ (&&), v (), ¬ (!)</code>
位运算符	BIT_AND, BIT_OR, BIT_XOR, BIT_NOT, SHL, SHR	<code>&, , ^, ~, <<, >></code>
赋值运算符	ASG (ASSIGN), AAS (PLUS_ASSIGN) ...	<code>=, +=, -=, *=, ...</code>
部分关键字	INT, IF, ELSE, WHILE, ...	<code>int,if,else,while,return,...</code>
部分分隔符	SCO (SEMI), CMA (COMMA), COLON, LPA, RPA, LBK, RBK, LBR, RBR	<code>;, \,, :, (,), [,], {, }</code>

注：等宽字体所示为工程通用符号，文本体的为本书用助记符。

对源程序预处理

- 删除空白和注释，处理宏和预处理指示项。

类别	Examples
comment	<code>/* ignored */</code>
preprocessor directive	<code>#include <foo.h></code>
	<code>#define NUMS 5, 6</code>
macro	<code>NUMS</code>
whitespace	<code>\t \n \b</code>

预处理结果

$\langle \text{源程序} \rangle \rightarrow (\langle \text{有效串} \rangle \langle \text{间隔} \rangle)^+ \langle \text{EOF} \rangle$

$\langle \text{有效串} \rangle \rightarrow (r_1 \mid \dots \mid r_n)^+$

$\langle \text{间隔} \rangle$ 可以是任意感兴趣的且不在 $\bigcup_{1 \leq i \leq n} \Sigma_i$ 中的符号。

后续：扫描器扫描一个 $\langle \text{有效串} \rangle$ 返回一个记号串，直到遇到 $\langle \text{EOF} \rangle$ ，依次将各记号串连接作为词法分析结果。

针对有效串的拆分与识别

逐一扫描<有效串>#直到EOF结束。

<有效串> $\rightarrow (r_1 \mid \dots \mid r_n)^+$

return21# $\rightarrow$ (ID, r)(ID, eturn21)✗

return21# $\rightarrow$ (RETURN, _)(NUM 21)✗

return21# $\rightarrow$ (ID, return21)✓

21return# $\rightarrow$ (NUM, 21)(RETURN, _)✓

2+3# $\rightarrow$ (NUM, 2)(ADD, _)(NUM, 3)?✓

2+3# $\rightarrow$ (NUM, 2)(NUM, +3)?✗

+15.07 $\rightarrow$ (ADD, _)(NUM, 15)(FLO, .07)✗

+15.07 $\rightarrow$ (NUM, +15)(FLO, .07)✗

+15.07 $\rightarrow$ (ADD, _)(FLO, 15.07)?✓

+15.07 $\rightarrow$ (FLO, +15.07)?✓

+N15.07 $\rightarrow$ (ADD, _)(ID, N)(FLO, 15.07)✗

+N15.07 $\rightarrow$ (ADD, _)(ID, N15)(FLO, .07)✓

一些情形可能不在实际语言中出现；但实际语言服从此原理。

- ① 剩余<有效串>的每一个前缀就是一个拆分方案，哪一个才是词法单元？
 - 前缀最大化原则
- ② 拆分出的前缀可能属于多个语言，其类别选哪个？
 - 事实优先级
- ③ 当前缀被识别为词法单元后，表示为一个词法记号，接着从剩余串中继续①和②直到剩余串为#结束。
- ④ 输出的记号串与<有效串>保持次序一致。可并行加速。

多语言联合DFA识别的概念

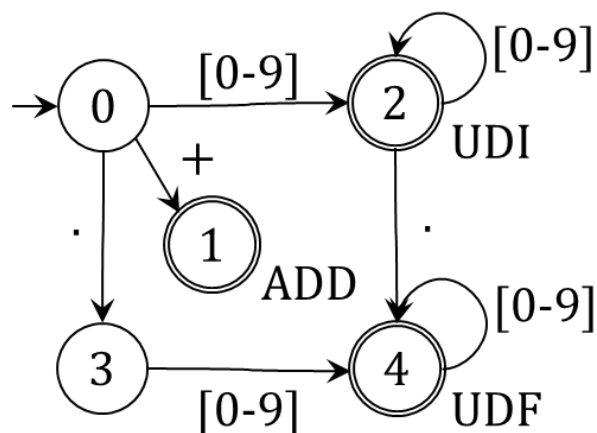
- 假定有 n 个语言依次定义为 $r_1, \dots, r_n$
- $\langle \text{有效串} \rangle \in L((r_1 | \dots | r_n)^+)$
- 为每个 r_i 构建 FA A_i , 初始状态为 q_i 。每个接受状态都带有标记 L_i 。
- 构建多语言联合NFA, 从初始状态 q_0 都有 ε 弧射出到 q_i 。
- 确定化为一个多语言联合DFA (σ -DFA), 它的每个接受状态的标记这样确定, 如果该状态包含NFA的多个接受状态, 那么选优先级最高的那个语言作为标记。
 - 即下标较小者优先级较高 (事实优先级)。

语言ADD、UDI和UDF联合DFA

王

消解冲突1: 事实优先级

消解冲突2: 前缀最大化原则



12+1.2#
(UDI, 12)(ADD, _)
(UDF, 1.2)

100#0.1#

$ADD = \{+\}$ $UDI = L([0-9][0-9]^*)$
 $UDF = L([0-9][0-9]^*.[0-9]^*+.[0-9][0-9]^*)$

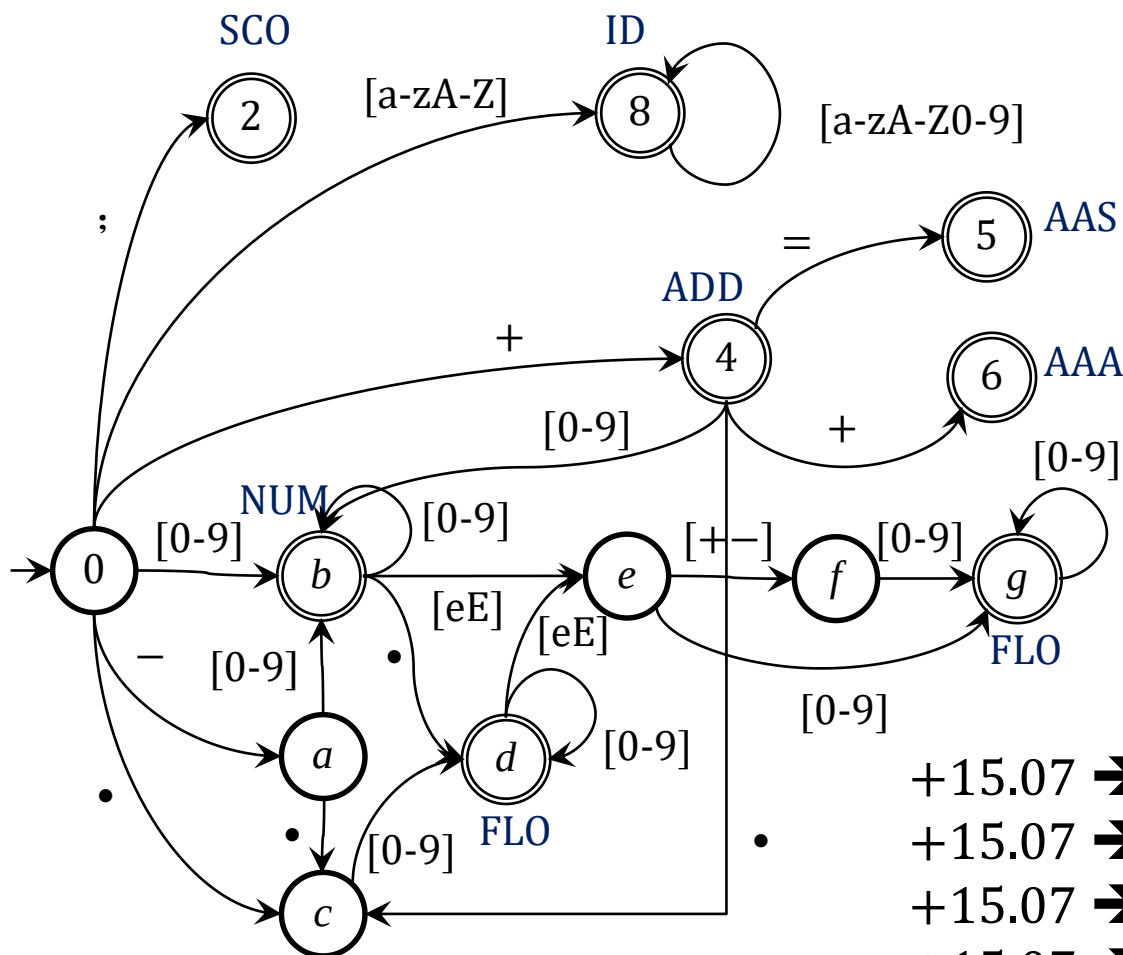
- 情形1: ADD、UDI与UDF不相交 无冲突
- 情形2: UDF包含UDI 状态2存在语言冲突

σ -DFA({ADD, UDI, UDF})的接受状态带有一个语言标记, 表明该状态接受的语言是唯一的, 这就将其语言划分为ADD、UDI和UDF了。

对情形2的讨论: 不可避免如+和++、ID和KEY等; 可用来简化设计。

示例：多语言联合DFA

王



return21# →

(ID, r)(ID, eturn21)✗
 (ID, return)(NUM 21)✗
 (ID, return21)✓

21return# →

(NUM, 21)(RETURN, _)✗
 (NUM, 21)(ID, return)✓

2+3# →

(NUM, 2)(NUM, +3)✓

2+3# →

(NUM, 2)(ADD, _)(NUM, 3)?

+15.07 → (ADD, _)(NUM, 15)(FLO, .07)✗

+15.07 → (NUM, +15)(FLO, .07)✗

+15.07 → (ADD, _)(FLO, 15.07)?✓

+15.07 → (FLO, +15.07)?✓

+N15.07 → (ADD, _)(ID, N)(FLO, 15.07)✗

+N15.07 → (ADD, _)(ID, N15)(FLO, .07)✓

σ -DFA原理

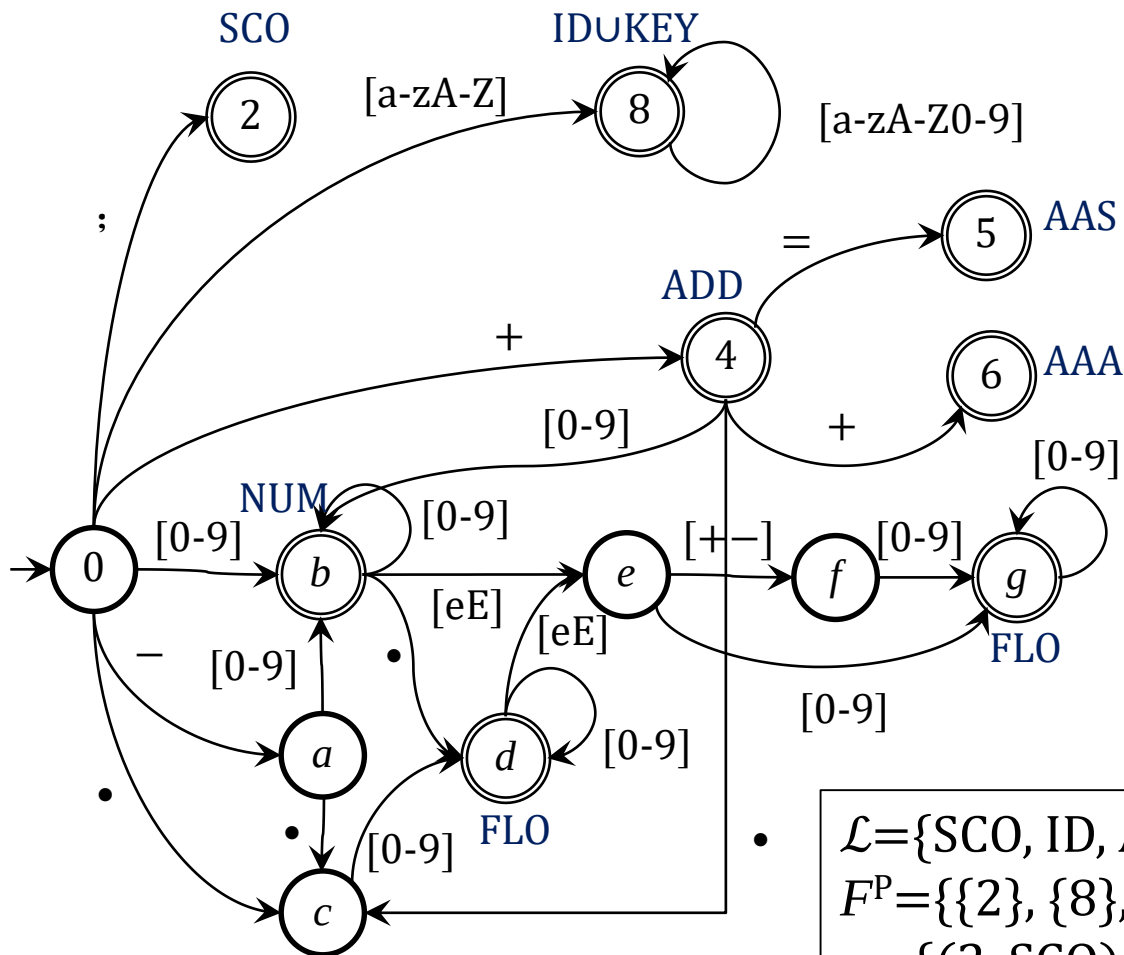
- 用 n 个语言的识别器 $r_1, r_2, \dots, r_n$ 同时判定一个串 x , 等价于用一个语言识别器 $r_1|r_2|\dots|r_n$ 来判定。
- 若 x 被接受, 那么 x 应该属于哪个语言呢? (按优先级决定)
- 构建联合DFA步骤: 已知 r_i 的NFA; 构建集成NFA; 转换为DFA; 对接受状态做语言标记。

σ -DFA原理（续）

- σ -DFA($A, \mathcal{L}$)是正则语言簇 $\mathcal{L}$ 上的多语言联合DFA，其中DFA $A=(Q, \Sigma, v, q_0, F)$ ， $\mathcal{L}$ 元素两两不相交。
- 满足 $\forall q \in F \cdot \exists L \in \mathcal{L} \cdot L(q) \subseteq L$ 且 $\forall L \in \mathcal{L} \cdot \exists q \in F \cdot L(q) \subseteq L$
- **定理4.1** σ -DFA($A, \mathcal{L}$)是正则语言簇 $\mathcal{L}$ 上的多语言联合DFA当且仅当 F 有一个与 $\mathcal{L}$ 一一对应的划分集 F^P ，其中 F 为 A 的接受状态集。
- 换句话说， F 的划分集 F^P 满足 $|F^P|=|\mathcal{L}|$ 且 $\forall P \in F^P \cdot (\cup q \in P \cdot L(q)) \in \mathcal{L}$ 。
- 也就是有函数 $\psi: F \rightarrow \mathcal{L}$
- $\psi(q)=L$ ，当且仅当 $\exists P \in F^P \cdot (q \in P \wedge L(q) \subseteq L)$

例： σ -DFA的构成

2018



$\mathcal{L} = \{\text{SCO, ID, AAS, ADD, AAA, NUM, FLO}\}$
 $F^P = \{\{2\}, \{8\}, \{4\}, \{5\}, \{6\}, \{B\}, \{D, G\}\}$
 $\psi = \{(2, \text{SCO}), (8, \text{ID}), (4, \text{ADD}), (5, \text{AAS}), (6, \text{AAA}), (B, \text{NUM}), (D, \text{FLO}), (G, \text{FLO})\}$

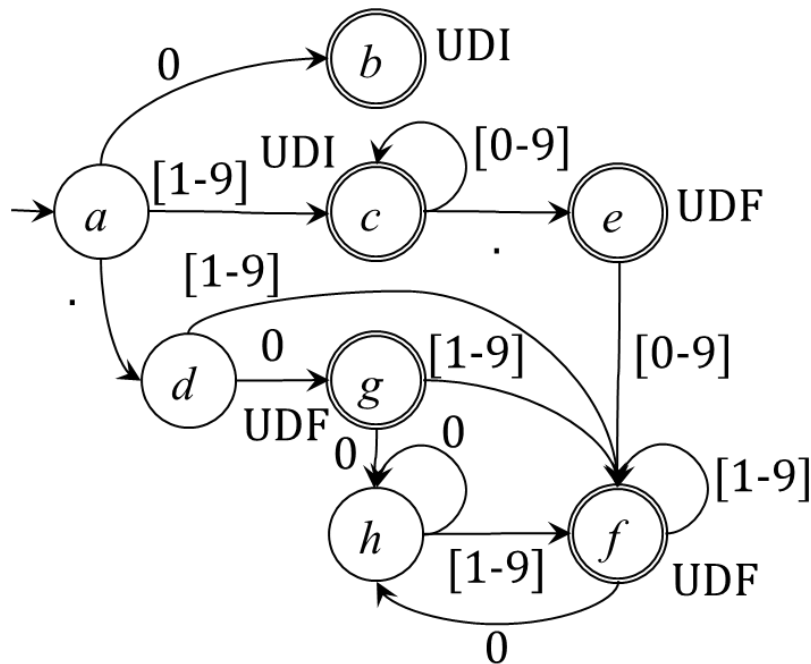
例: σ -DFA($\{\text{UDI}, \text{UDF}\}$)

$\langle \text{udi} \rangle \rightarrow 0 \mid [1-9][0-9]^*$

$\langle \text{tail} \rangle \rightarrow [1-9] \mid [0-9]^*[1-9]$

$\langle \text{udf} \rangle \rightarrow \langle \text{udi} \rangle \backslash . \langle \text{tail} \rangle \mid \langle \text{udi} \rangle \backslash . \mid \langle \text{udi} \rangle \backslash . 0 \mid \backslash . \langle \text{tail} \rangle \mid 0 \backslash . \langle \text{tail} \rangle$

语言 $\text{UDI} = L(\langle \text{udi} \rangle)$ 和 $\text{UDF} = L(\langle \text{udf} \rangle)$ 都是正则语言，存在 σ -DFA。



$$F^P = \{\{b, c\}, \{e, f, g\}\}$$

$$\text{UDI} = \{w \in \Sigma \mid \tilde{v}(a, w) \in \{b, c\}\}$$

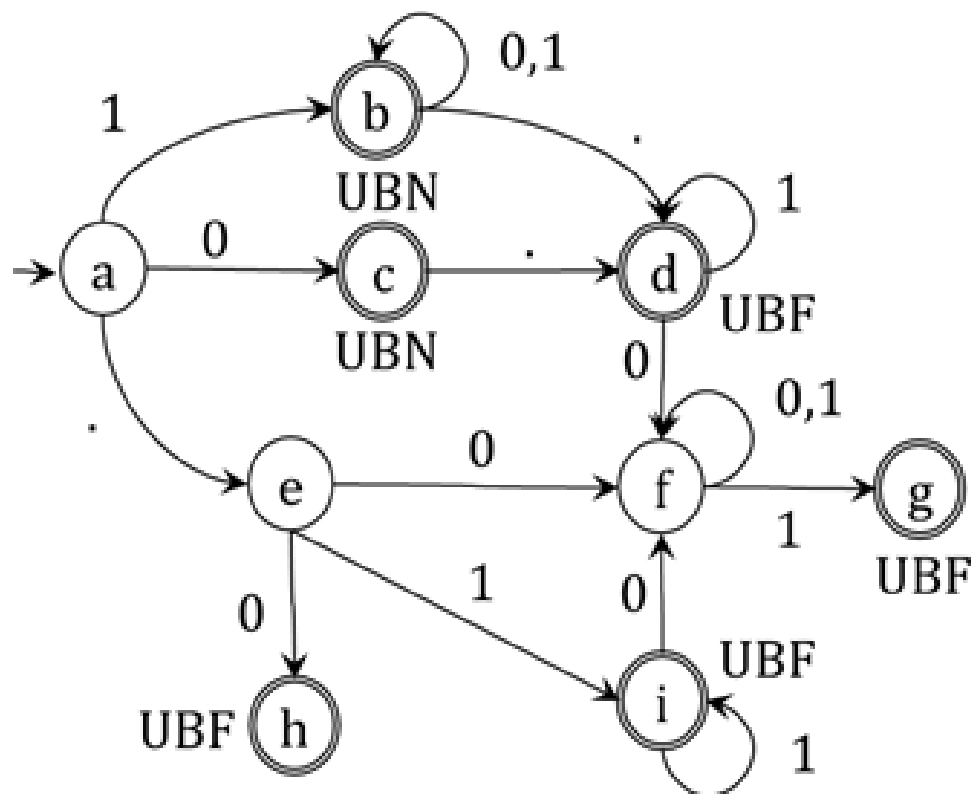
$$\text{UDF} = \{w \in \Sigma \mid \tilde{v}(a, w) \in \{e, f, g\}\}$$

$$\psi = \{(b, \text{UDI}), (e, \text{UDF}), (c, \text{UDI}), (f, \text{UDF}), (g, \text{UDF})\}$$

$$\text{UDI} \cap \text{UDF} = \emptyset$$

例： σ -DFA($\mathcal{L}$)

- $\mathcal{L} = \{\text{UBN}, \text{UBF}\}$
- $F^P = \{\{b, c\}, \{d, g, h, i\}\}$
- $\psi = \{(b, \text{UBN}), (d, \text{UBF}), (c, \text{UBN}), (g, \text{UBF}), (h, \text{UBF}), (i, \text{UBF})\}$



构建 σ -DFA($\mathcal{L}$)

- 构建语言簇 $\mathcal{L}$ 上的联合DFA，其中 $\mathcal{L}$ 的元素两两不相交。
- 用到一个松弛的有序语言簇 $\mathbb{L}$ ，二者元素个数相同，满足， $\mathcal{L} = \{L_1, \dots, L_n\}$, $L_i = \mathbb{L}[i] \setminus \mathbb{L}[i-1] \setminus \dots \setminus \mathbb{L}[1]$ ，可见 L_i 可以被放大至 $\mathbb{L}[i]$ ，从而获得构建上的便利。
- 若 $w \in \mathbb{L}[i] \setminus \dots \setminus \mathbb{L}[1]$ 且 $w \notin \mathbb{L}[k]$, $k < i$ ，那么 w 的词法类别为 $L = \mathbb{L}[i] \setminus \dots \setminus \mathbb{L}[1]$ ， $L \in \mathcal{L}$ ，但对任意 $j > i$ ， w 也可能属于 $\mathbb{L}[j] \setminus \dots \setminus \mathbb{L}[1]$ 但是不能改变它的词法类别 L 。这里使用了事实优先级原则，即有序集合 $\mathbb{L}$ 下标较小者的优先级较高。

算法4.1 构造 σ -DFA ($\mathcal{L}$)

王

输入: n 个正则语言的有序集合 $\mathbb{L}$, $n=|\mathcal{L}|$,

满足 $\{\mathbb{L}[i] \setminus \mathbb{L}[i-1] \setminus \dots \setminus \mathbb{L}[1] \mid i=1, \dots, n\} = \mathcal{L}$

输出: σ -DFA $((Q, \Sigma, v_D, q_0, F), \mathcal{L})$ 且有满射映射

$\{(q, L) \mid q \in F, L = \mathbb{L}[\text{mark}(q)] \setminus \mathbb{L}[\text{mark}(q)-1] \setminus \dots \setminus \mathbb{L}[1]\}$

(1) 为每个 $\mathbb{L}[i]$, $1 \leq i \leq n$, 构造有穷自动机FA $A_i = (Q_i, \Sigma_i, v_i, q_i, F_i)$, 且

$Q_i \cap Q_j = \emptyset, i \neq j$;

(2) 对于 $i=1, \dots, n$, 令 $\forall q \in F_i \cdot \text{mark}[q]=i$ 且

$\forall q \in Q_i \setminus F_i \cdot \text{mark}[q]=\infty$;

(3) 构建集成NFA $N = (Q_N, \Sigma, v_N, q_0, F_N)$, 其中

$Q_N = \{q_0\} \cup \bigcup i \cdot Q_i$; $q_0 \notin \bigcup i \cdot Q_i$; $\Sigma = \bigcup i \cdot \Sigma_i$;

$v_N = \{(q_0, q_i, \varepsilon) \mid 1 \leq i \leq n\} \cup \bigcup i \cdot v_i$; $F_N = \bigcup i \cdot F_i$; $\text{mark}[q_0] = \infty$;

算法4.1构造 σ -DFA ($\mathcal{L}$) (续)

2018

输入: n 个正则语言的有序集合 $\mathbb{L}$, $n=|\mathcal{L}|$,

满足 $\{\mathbb{L}[i] \setminus \mathbb{L}[i-1] \setminus \dots \setminus \mathbb{L}[1] \mid i=1, \dots, n\} = \mathcal{L}$

输出: σ -DFA $((\mathbb{Q}, \Sigma, v_D, \mathbb{Q}_0, \mathbb{F}), \mathcal{L})$ 且有满射映射

$\{(\mathbb{q}, L) \mid \mathbb{q} \in \mathbb{F}, L = \mathbb{L}[\text{mark}(\mathbb{q})] \setminus \mathbb{L}[\text{mark}(\mathbb{q})-1] \setminus \dots \setminus \mathbb{L}[1]\}$

(3) 构建集成NFA $N=(Q_N, \Sigma, v_N, q_0, F_N)$, 其中

$Q_N = \{q_0\} \cup \bigcup i \cdot Q_i$; $q_0 \notin \bigcup i \cdot Q_i$; $\Sigma = \bigcup i \cdot \Sigma_i$;

$v_N = \{(q_0, q_i, \varepsilon) \mid 1 \leq i \leq n\} \cup \bigcup i \cdot v_i$; $F_N = \bigcup i \cdot F_i$; $\text{mark}[q_0] = \infty$;

(4) 采用子集法将NFA N 转换为DFA $A=(\mathbb{Q}, \Sigma, v_D, \mathbb{Q}_0, \mathbb{F})$;

令 $\forall \mathbb{q} \in \mathbb{F} \cdot \text{mark}[\mathbb{q}] = \min_{q \in \mathbb{q}} \text{mark}(q)$;

(5) 返回 σ -DFA $(\mathcal{L}) = A$ 且有满射映射

$\{(\mathbb{q}, L) \mid \mathbb{q} \in \mathbb{F}, L = \mathbb{L}[\text{mark}(\mathbb{q})] \setminus \mathbb{L}[\text{mark}(\mathbb{q})-1] \setminus \dots \setminus \mathbb{L}[1]\}$ 。

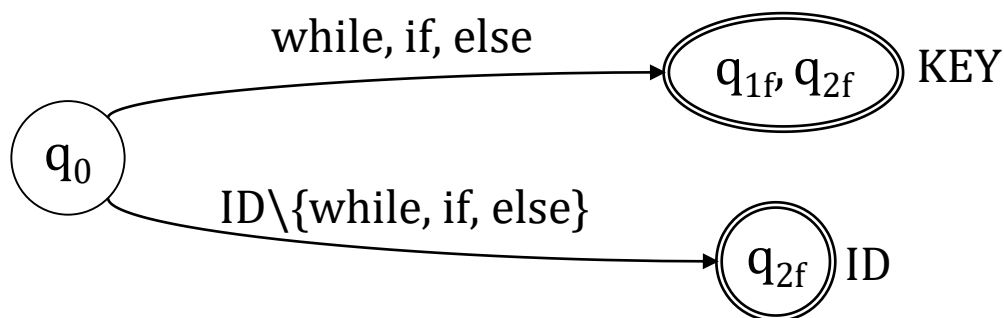
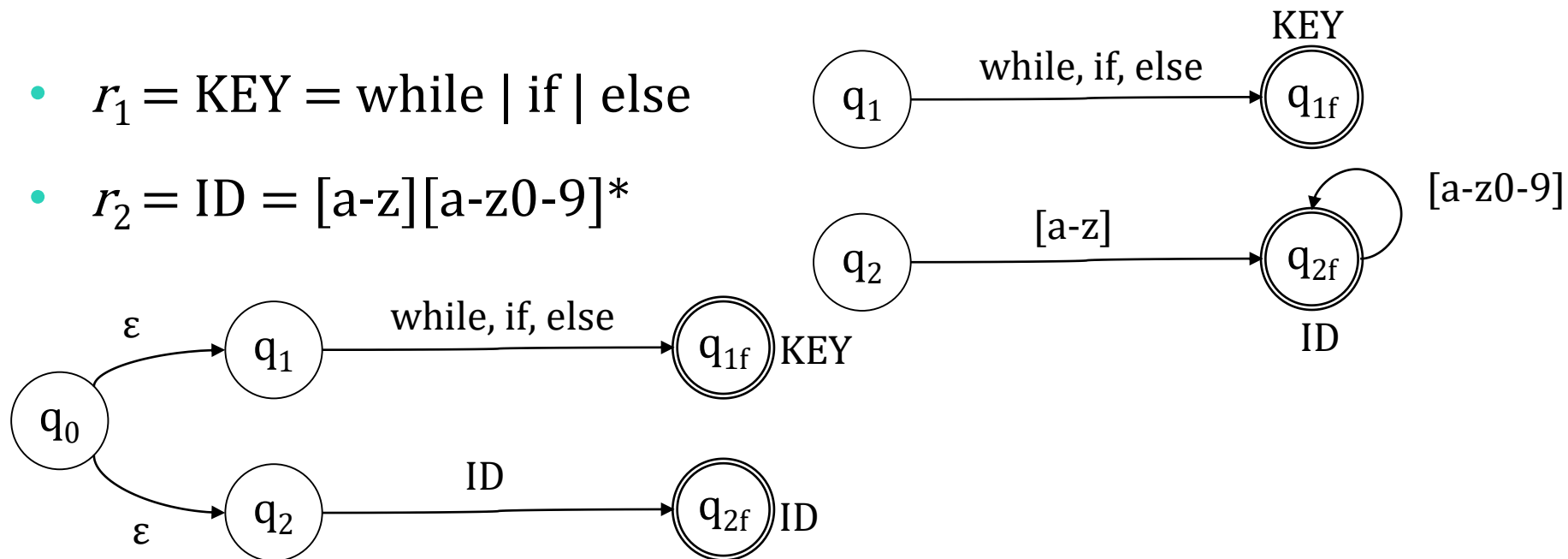
例：标识符与关键字

石川

- 构造算法的输入中下标较小者的优先级较高

- $r_1 = \text{KEY} = \text{while} \mid \text{if} \mid \text{else}$

- $r_2 = \text{ID} = [\text{a-z}][\text{a-z0-9}]^*$



4.4 词法分析器设计

- 词法记号设计
- 确定好各个词法单元种属，构建 σ -DFA
- 借用字符流控制功能编码实现该 σ -DFA
- σ -DFA基于前缀最大化匹配原则进行识别
- 实现扫描器：
 - 硬编码实现
 - 基于转移表的实现
- σ -DFA用于指令选择
 - 大作业（1/2）
- 自动生成扫描器（4.5节）

词法单元的多场景呈现

源程序	词法分析		语法、语义分析	
文本	文本	计算	文本	计算
子串	词法记号	存储	终结符	lexeme
100	$\langle \text{NUM}, 100 \rangle$	数值存储	$\mathbf{i} \langle 100 \rangle$	100
fact	$\langle \text{ID}, \text{fact} \rangle$	符号表存储	$\mathbf{d} \langle \text{fact} \rangle$	fact
if	$\langle \text{IF}, _ \rangle$	字符串存储	$\text{if} \langle \text{if} \rangle$	if
+	$\langle \text{PLUS}, _ \rangle$	字符串存储	+	+
+=	$\langle \text{AAS}, _ \rangle$	字符串存储	+=	+=
"Hi!"	$\langle \text{STR}, \text{"Hi!"} \rangle$	字符串存储	$\mathbf{s} \langle \text{"Hi!"} \rangle$	"Hi!"

超前搜索

- 前缀最大化匹配原则能让 σ -DFA尽可能读入符号。
- 当 σ -DFA继续运行到拒绝的时候，就需要回退到最近一次在接受状态的地方并返回结果。从那以后多读入的字符都要退回给输入流。
- 接下来如果输入流还有，则从初始状态开始，对剩余串再次运行 σ -DFA识别下一词法单元。
- 在存储受限情况下可采用乒乓缓冲：
 - 每个缓冲区大小不小于最大超前搜索长度。
 - 保持一个缓冲区既是另一个的前驱又是后继。

词法错误处理

- 如果输入串的前缀都不是合法记号，则遇到一个词法错误，报错，这是真错误。
- 跳过一些符号直到找到合法前缀，会找到更多错误，不一定是真错误。
- 一般会给出错误的位置。

基于硬编码的词法分析程序

- 例4.6 用C语言写出一个词法分析器，可识别的词法种属有：
无符号整数（UDI）；不含下划线的ID；运算符ADD、ASG、AAS；关键字INT；分隔符LPA、RPA、LBK、RBK、SCO。还包括默认的EOF和ERR。示例将输入串 `int x;x+=b[1]++2` 识别为词法记号串：(INT,_) (ID,x) (SCO,_) (ID,x) (AAS,_) (ID,b) (LBK,_) (UDI,1) (RBK,_) (ADD,_) (ADD,_) (UDI,2)

设计符号表保存标识符信息

- 符号表登记项组织为一个单链表结构，结点（**SymbolNode**）是一个标识符的登记项，结点的**name**域值为标识符名字。
- 一个登记项除了标识符名字外的信息在词法分析阶段不关心
- 函数**SymbolNode* bind(Scanner *scanner, const char *name)**将新的标识符加入符号表，返回该标识符的登记项入口。
- 代码中需要判断符号串是不是关键字：
int is_keyword(const char *str)

定义 $\mathcal{L}$ 类型为枚举类型

石华

- `typedef enum {UDI=1, ID, ADD, ASG, AAS, INT, LPA, RPA, LBK,`
- `RBK, SCO, KIND_EOF, ERR} TokenKind;`
- 其中包含默认的EOF和ERR
- 允许索引访问各成员

定义词法记号为结构类型

- 定义词法记号为结构类型，有种属`kind`、词法单元文本`lexeme`、词法单元数值`value`、标识符的登记项`entry`四个分量。在使用 `Token c` 变量时允许与`c.kind`无关的分量未初始化。`getn(c)`、`getv(c)`分别返回词法单元的文本和数值。
- `#define MAX_TOKEN_LEN 100`
- `typedef struct {`
- `int kind;` `char lexeme[MAX_TOKEN_LEN];`
- `int value;` `SymbolNode *entry; } Token;`

初始化扫描器Scanner

- 定义及初始化扫描器的全局设置，为当前剩余串和默认的符号表两个分量组成**Scanner**结构。通过当前剩余串指针**current**也同时给出了当前输入符号。而当前符号表仅使用它的表项链表。初始化**Scanner**需要给定一个源程序符号串和一个符号表。

```
typedef struct {
    const char *current;
    SymbolNode *symTable;
} Scanner;

void initScanner(Scanner *scanner, const char *source) {
    scanner->current = source;
    initSymbolTable(scanner);
}
```


主扫描函数

- 主扫描函数`scanner()`每调用一次返回一个词法记号，具体是从剩余串的前缀中识别出一个词法单元并返回，同时更新剩余串。使用标准库函数完成相关的判定，如`isspace(c)`、`isdigit(c)`、`isalpha(c)`、`isalnum(c)`依次判字`c`为空白字符、数字、字母、字母数字。另外还用到`snprintf`函数格式化字符串并赋值。

```
Token scanner(Scanner *scanner) {
    Token token;
    // 跳过空白字符
    while (isspace(*scanner->current)) scanner->current++;
    // 检查是否结束
    if (*scanner->current == '\0') {        token.kind = KIND_EOF;
        strcpy(token.lexeme, "EOF");        return token; }
}
```

无符号整数的处理

```
if (isdigit(*scanner->current)) {
    const char *start = scanner->current;
    while (isdigit(*scanner->current)) scanner->current++;
    int len = scanner->current - start;
    strncpy(token.lexeme, start, len);
    token.lexeme[len] = '\0';
    token.kind = UDI;
    token.value = atoi(token.lexeme);
    return token; }
```

标识符、关键字的处理

邵伟

```
if (isalpha(*scanner->current)) {  
    const char *start = scanner->current;  
    while (isalnum(*scanner->current)) scanner->current++;  
    int len = scanner->current - start;  
    strncpy(token.lexeme, start, len);  
    token.lexeme[len] = '\0';  
    // 检查是否是关键字  
    if (is_keyword(token.lexeme)) token.kind = INT;  
    else {  
        token.kind = ID;  
        token.symbol = bind(scanner, token.lexeme);    }  
    return token; }  
}
```

单字符和多字符Token的处理

王

```
char c = *scanner->current;
scanner->current++;
switch (c) {
    case '+':
        if (*scanner->current == '=') {
            scanner->current++;
            token.kind = AAS;
            strcpy(token.lexeme, "+=");
        } else {
            token.kind = ADD;
            strcpy(token.lexeme, "+");
        }
        break;
    case '=': token.kind = ASG; strcpy(token.lexeme, "="); break;
    case '[': ...           case ']': ...           case ';': ...
```

错误处理

- 当词法分析器遇到无法识别的字符时，它会创建一个错误Token，这行代码就是格式化错误信息字符串并保存到Token的lexeme字段中。

default:

```

        token.kind = ERR;
        snprintf(token.lexeme, MAX_TOKEN_LEN, "非法字符
'%c'", c);
        break;
    }
    return token;
}

```

硬编码小结

- 上述程序代码显示词法分析器的完整结构。
- 虽然尚有词法单元不能被识别，但可以局部性地扩展代码加入，直至形成实用的词法分析器。
- 注意，有效扩展的前提是仔细地设计 σ -DFA。

基于转移表的词法分析器

- 将 σ -DFA的状态转移表实现为一个二维数组`Delta[]`，以当前状态和当前输入符号为索引，返回转移后的状态。
- 在`Delta[]`中包含接受状态，特别使用`Accept[]`表复制它们以便检测。
- 在`Delta[]`中还包括错误状态，使用`error()`检测并进行错误处理。
- 采用如下的代码模板完成最基本的基于转移矩阵的词法扫描过程，即假定有效串最多一个前缀是词法单元，这是使用前移表`Advance[]`表的原因。前移表定义是否消耗当前字符。

```
state = start state;  
ch = next input char;  
while not Accept[state] and not error(state) do  
    newstate = Transition_Table[state, ch];  
    if Advance[state, ch] then ch = next input char;  
    state = newstate;  
end while;  
if Accept[state] then accept;
```


词法分析任务

- 例4.6 $\Sigma_{\text{src}} = \{0, 1, a, b, c, +, =, ;, \#, _ \}$, 对于 $\text{UBI} = \{b \mid b \in \{0, 1\}^* \wedge 1 \leq |b| \leq 16\}$ 、 $\text{ID} = \{w \mid w \in \{a, b, c\}^* \wedge 1 \leq |w| \leq 16\}$ 、以及 ADD, ASG, AAS, INT, SCO 和 ERR 等多个语言, 构建基于转移表的简短扫描器。

定义有用的类型

- 示例表格驱动的扫描过程，简化了其它功能。首先没有使用符号表，也没有产生整数值，所以词法记号Token类型仅仅由种属和词法单元文本两部分组成，与文本形式的二元组一致。

```
typedef enum { UBI, ID, ADD, ASG, AAS, INT, SCO, ERR }
TokenKind;

typedef struct { const char *cur; } Scanner;

typedef struct { TokenKind kind; char lexeme[16]; } Token;

typedef enum { T_=0, ST, S1, S2, S3, SH, S4, S5, S6 } State;
// Q=[0,8]
```

初始化有用的表格

- 转移表Delta[9][10]和Advance表A[9][10]都是|Q|行， $|\Sigma_{src}|$ 列的表格，Accept[9]是长度为|Q|的一维表格，使用Q的元素作为这些表格第一维的索引值，使用 Σ_{src} 的元素作为这些表格第二维的索引值，并依照 σ -DFA($\mathcal{L}$)的转移表来初始化这些表格。

State Delta[9][10] = {	int Advance[9][10] = {
{0}, //S_=0	{0},
{2, 2, 3, 3, 3, 4, 6, 7, 5, 0}, //ST=1	{1, 1, 1, 1, 1, 1, 1, 1, 1, 0},
{2, 2, 0, 0, 0, 0, 0, 0, 0, 0}, //S1=2	{1, 1, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 3, 3, 3, 0, 0, 0, 0, 0}, //S2=3	{0, 0, 1, 1, 1, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 8, 0, 0, 0}, //S3=4	{0, 0, 0, 0, 0, 0, 1, 0, 0, 0},
{5, 5, 5, 5, 5, 5, 5, 5, 5, 5} //SH=5	{1, 1, 1, 1, 1, 1, 1, 1, 1, 1}};
} //S4=6, S5=7, S6=8; 行默认全0	

int Accept[9] = {0, 0, 1, 1, 1, 0, 1, 1, 1}; // S1, S2, S3, S4, S5, S6

定义实用函数

- 将输入符号 (Σ_{src} 成员) 转换为索引值, 用于表格第一维度元素的索引。接着初始化源程序符号串为剩余串。**nxt**函数读入当前输入符号返回之或返回结束符。**peek**函数查看当前输入符号。

```
int getCharClass(char ch) { if (ch == '0') return 0; if (ch == '1') return 1;
    if (ch == 'a') return 2; if (ch == 'b') return 3;
    if (ch == 'c') return 4; if (ch == '+') return 5;
    if (ch == '=') return 6; if (ch == ';') return 7;
    if (ch == '#') return 8; return 9; }
```

```
void initS(Scanner *s, const char *src) { s->cur = src; }
char nxt(Scanner *s) { return *s->cur ? *s->cur++ : 0; }
char peek(Scanner *s) { return *s->cur; }
```

主扫描函数

- 主扫描函数，返回一个词法记号。符号#表示分隔符，也当作空白字符使用。符号_表示其它不属于 Σ_{src} 的字符，也作为字符串结尾'\0'和文件结尾，表示输入流结束，返回一个种属为ERR的名字为END的记号。

```
Token scanner(Scanner *s) {
    Token t; State st=ST; char ch; int idx=0;
    while(peek(s) == '#') nxt(s);  \跳过空白字符
    if(peek(s) == '_' || !peek(s)) {
        t.kind=ERR; strcpy(t.lexeme,"END"); return t;
    }
}
```

```

while(!Accept[st]) {
    ch = peek(s);
    int cc = getCharClass(ch);
    State ns = Delta[st][cc];
    if(st==S2 && idx==2 && t.lexeme[0]=='i' &&
        t.lexeme[1]=='n' && ch=='t') {    \ \ 扫描关键字
        char next = s->cur[1];
        if(next=='_' || !next || !(next=='a' || next=='b' || next=='c')) {
            nxt(s);
            t.lexeme[idx++]='t';
            t.lexeme[idx]=0;
            t.kind=INT;
            return t;
        }    } //如果有多个关键字，可扩展为查表方式

```

```

if(ns==T_) {
    t.kind=ERR;
    t.lexeme[0]=nxt(s);
    t.lexeme[1]=0;
    return t;
}

if(Advance[st][cc] && ch!='#' && ch!='_') {
    t.lexeme[idx++] = nxt(s);  \\读入当前符号
} else if(Advance[st][cc]) {
    nxt(s);
}

st = ns;
}

```

由接受状态得出词法单元的种属

```
t.lexeme[idx]=0;
switch(st) { \由接受状态得出词法单元的种属
    case S1: t.kind=UBI; break;
    case S2:
        if(strcmp(t.lexeme,"int")==0) t.kind=INT;
        else t.kind=ID;
        break;
    case S3: t.kind=ADD; break;
    case S4: t.kind=ASG; break;
    case S5: t.kind=SCO; break;
    case S6: t.kind=AAS; break;
    default: t.kind=ERR; break;
}
return t;
```


表驱动小结

- 理论上表格驱动的词法分析程序代码与Delta[]的内容没有关系，虽然该代码的实际执行流程与Delta[]内容有关，而且这是这一方法的一个优点，但是该表格一般是稀疏的，用其驱动词语分析程序存在空间上的开销。稀疏矩阵的压缩表示可以减少这种开销。

基于 σ -DFA的指令选择

2016

- 大作业（1/2）：基于中间语言与指令模板的 σ -DFA构建及鲲鹏环境验证。
 - 理论构建：根据设计的中间语言QTAC和指令模板，构建 σ -DFA，用于指导代码生成。
 - 程序实现：编写程序实现上述DFA逻辑，将中间代码转换为特定架构的汇编代码。
 - 环境验证：利用Fact程序（或类似的测试驱动/验证工具）生成测试用例，得出ARM汇编代码。
 - 硬件运行：在华为鲲鹏（ARM架构）环境下完成汇编、链接并运行，验证代码生成的正确性。

指令模板局部

2018

GOTO l

GOTO q_s

LABEL l

IF $q_s \langle \text{rop} \rangle q_t$ THEN l_1 ELSE l_2 ; LABEL l_2

IF $q_s \langle \text{rop} \rangle q_t$ THEN l_1 ELSE l_2 ; LABEL l_1

IF $q_s \langle \text{rop} \rangle q_t$ THEN l_1 ELSE l_2

IF $q_s \langle \text{rop} \rangle k$ THEN l_1 ELSE l_2 ; LABEL l_2

IF $q_s \langle \text{rop} \rangle k$ THEN l_1 ELSE l_2 ; LABEL l_1

IF $q_s \langle \text{rop} \rangle k$ THEN l_1 ELSE l_2

$t = q_s + k$; $q_d = M[t^{\text{last}}]$

$q_d = M[q_s]$

$q_d = M[l]$

$t = q_s + k$; $M[t^{\text{last}}] = q_t$

$M[q_s] = q_t$

$M[l] = q_s$

$q_d = q_s \langle \text{aop} \rangle q_t$

$q_d = q_s$

$q_d = q_s \langle \text{aop} \rangle k$

$q_d = k$

B l

BR X_s

l :

CMP X_s, X_t ; $\langle \text{Jrop} \rangle l_1$; l_2 :

CMP X_s, X_t ; $\langle \text{Jrop}' \rangle l_2$; l_1 :

CMP X_s, X_t ; $\langle \text{Jrop} \rangle l_1$; B l_2

CMP $X_s, \#k$; $\langle \text{Jrop} \rangle l_1$; l_2 :

CMP $X_s, \#k$; $\langle \text{Jrop}' \rangle l_2$; l_1 :

CMP $X_s, \#k$; $\langle \text{Jrop} \rangle l_1$; B l_2

LDR $X_d, [X_s, \#k]$

LDR $X_d, [X_s]$

LDR $X_d, =l$; LDR $X_d, [X_d]$

STR $X_t, [X_s, \#k]$

STR $X_t, [X_s]$

LDR $\langle \text{Xcrs} \rangle, =l$; STR $X_s, [\langle \text{Xcrs} \rangle]$

$\langle \text{Xaop} \rangle X_d, X_s, X_t$

MOV X_d, X_s

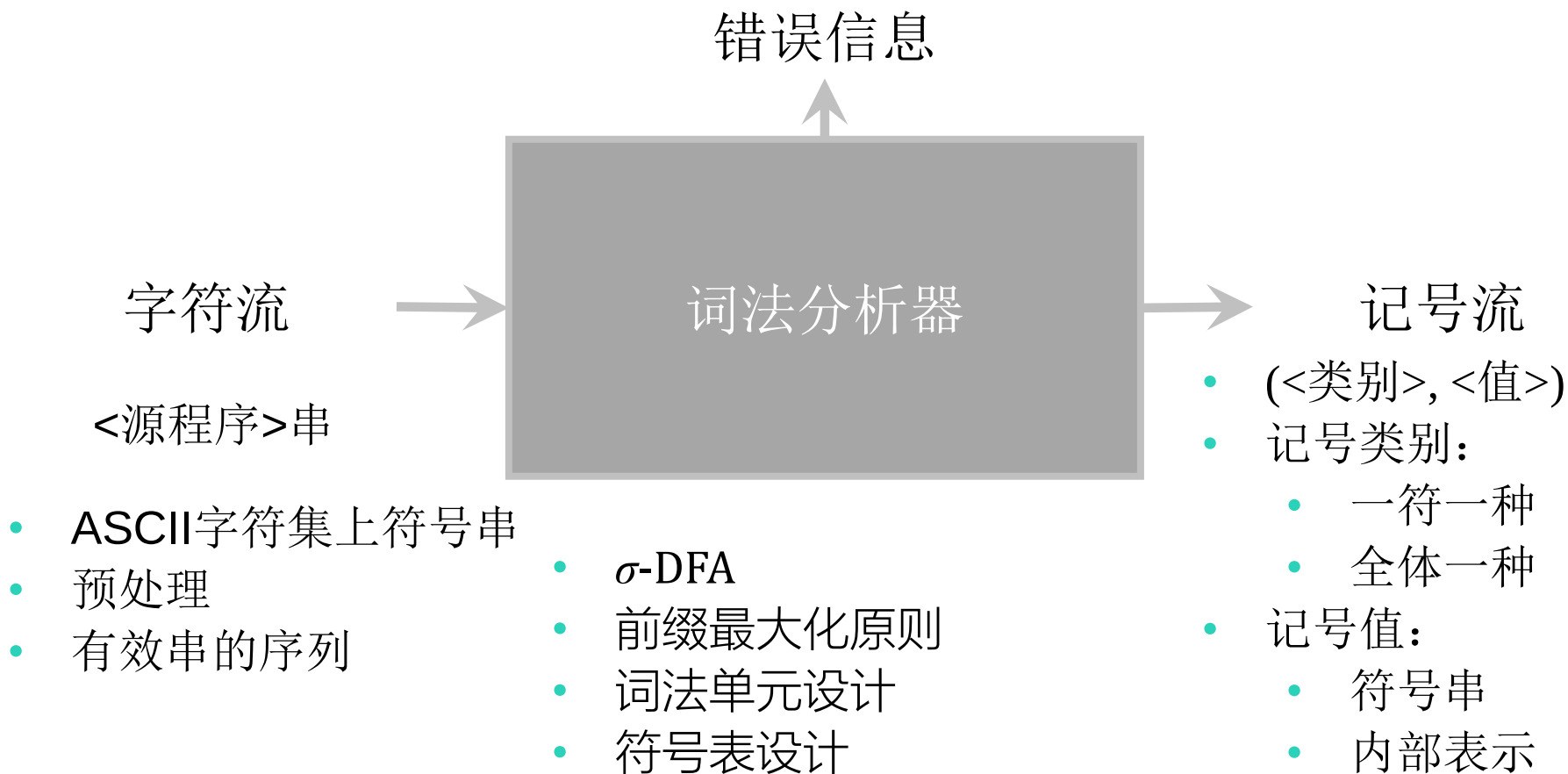
$\langle \text{Kaop} \rangle X_d, X_s, \#k$

MOV $X_d, \#k$

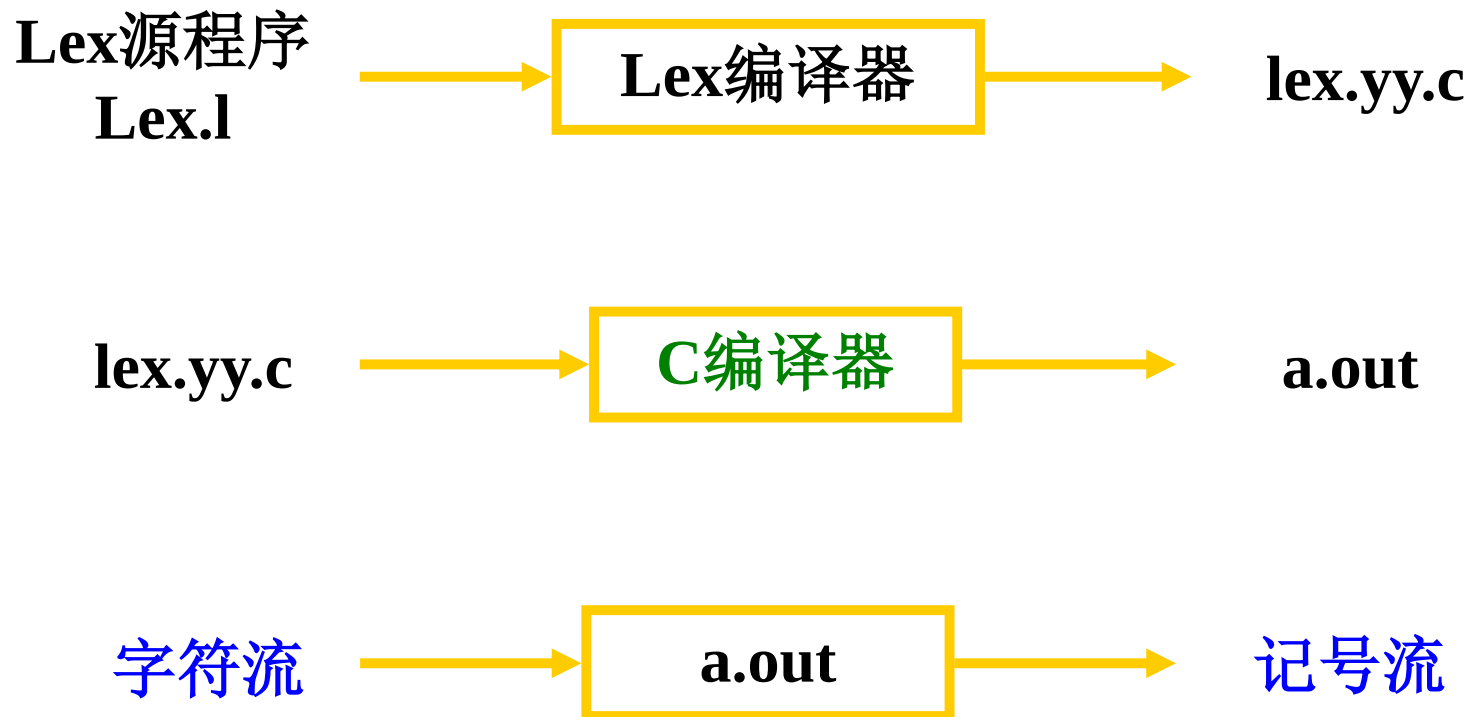
大作业（1/2）具体安排

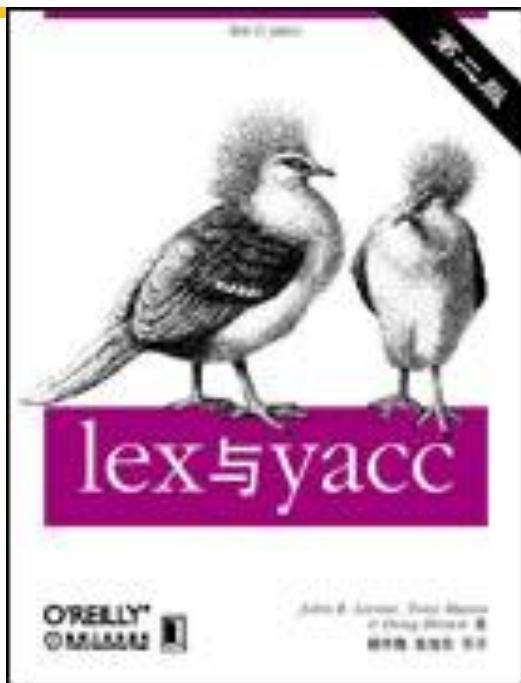
- 任务依据：实现指导书
- 实验成果提交：实验报告
- 实验指导书于本周二发布
- 报告提交日期：暂定第7周周末

小结：词法分析器的设计与实现



4.5 词法分析器的自动生成





《lex与yacc（第二版）》

作者: [John R. Levine](#), [Tony Mason](#), [Doug Brown](#) 著

杨作梅,张旭东,等 译

出版: 2003年1月

书号: 7-111-10721-7

页数: 392

定价: 45.00元

Lex程序结构

邵伟

definitions

变量、常量和正规定义

%%

rules

r_1 {Actions₁}

...

%%

r_n {Actions_n}

user code

符号表代码、其它函数等

delim	[\t\n]
ws	{delim}+
letter	[A-Za-z]
digit	[0-9]
id	{letter}({letter} {digit})*
number	{digit}+(\.{digit}+)?(E[+\-]?{digit}+)?
%%	
{ws} {}	
if	{return(IF);}
then	{return(THEN);}
else	{return(ELSE);}
{id}	{yylval=install_id();return(ID);}
{number}	{yylval=install_num();return(NUMBER);}
"<"	{yylval=LT;return(RELOP);}
...	
%%	
install_id()	{ 单词装入符号表并返回指针; }
...	

本章小结

- 知识点：多语言联合DFA、最大前缀匹配原理、一符一种记号、全体一种记号，编程语言常见记号类别
- 作业 p121 习题4.2、
- 大作业（1/2）：基于中间语言与指令模板的 σ -DFA构建及鲲鹏环境验证（待发布）